

PA1 Report: PQ-SHP (Part 3)

Diogo Jesus¹, Nuno Nogueira²

¹ n° 59969.

² n° 60599.

Keywords: Post-quantum cryptography, ML-DSA, ML-KEM, hybrid handshake, FIPS 204, FIPS 203, forward secrecy, key encapsulation mechanism, digital signatures, SHP.

1 Part 3: Post-Quantum SHP

1.1 Overview

Part 3 extends SHP with post-quantum algorithms to resist attacks from a cryptographically relevant quantum computer. Two variants are provided:

- **Part 3a (Hybrid):** ML-DSA-65 replaces ECDSA for authentication. Ephemeral ECDH is retained for key agreement.
- **Part 3b (Full PQ):** ML-DSA-65 for authentication and ML-KEM-768 for key encapsulation, replacing ECDH entirely.

The RTSSP framing layer, cipher suite negotiation, and key derivation are unchanged from Part 2.

1.2 Algorithm Choices

ML-DSA-65 (FIPS 204, formerly Dilithium): NIST Level 3 signature algorithm used by both variants, with a 1952-byte public key and 3309-byte signature. Keys are generated by `keytool` on Java 25 (JEP 497) and loaded from PKCS12 keystores. Signing and verification use `Signature.getInstance("ML-DSA-65")` via the JDK native provider.

ML-KEM-768 (FIPS 203, formerly Kyber): IND-CCA2 secure KEM at NIST Level 3, used in Part 3b for key establishment. Key pairs are generated ephemeral per handshake inside `PqShpCryptoSpec` and discarded after decapsulation, preserving the forward-secrecy guarantee from Part 2.

1.3 Protocol Flow

The Part 3a (Hybrid) flow is identical to Part 2 SHP with ML-DSA-65 substituted for ECDSA so the message structure is unchanged.

Part 3b (Full PQ) replaces the ECDH key exchange with ML-KEM encapsulation:

1. **PQ_CLIENT_HELLO:** X.509 certificate, ML-KEM-768 public key (1184 bytes), ordered suite list, 16-byte nonce, ML-DSA-65 signature.
2. **PQ_SERVER_HELLO:** X.509 certificate, ML-KEM-768 ciphertext (1088 bytes) encapsulating the shared secret, selected suite name, 16-byte server nonce, `clientNonce+1`, ML-DSA-65 signature.

3. **CSSP**: `encrypt(serverNonce+1 | udpPort)`, optionally with an integrity proof, using the session key derived from the KEM shared secret.

Both sides then call `CipherSuiteFactory.fromConfig(suite, sharedSecret)`, identical to Part 2.

1.4 Implementation

New classes live under `shp/pq/` and `crypto/`, leaving Parts 1 and 2 untouched.

- `MlDsaSignature`: implements `DigitalSignature` via the JDK native ML-DSA provider (JEP 497).
- `MlKemEncapsulation`: wraps Bouncy Castle 1.84 ML-KEM key generation, encapsulation, and decapsulation.
- `PqShpCryptoSpec`: per-session bundle of ML-DSA signing key, ephemeral ML-KEM key pair, and X.509 certificate.
- `HybridShpServer` / `HybridShpClient`: reuse the Part 2 protocol classes with `MlDsaSignature` injected in place of `EcDsaSignature`.
- `PqShpServer` / `PqShpClient`: drive the Full PQ protocol classes (`PqShpServerProtocol` / `PqShpClientProtocol`).

PKI stores are generated with `scripts/generate-pq-stores.sh`, which calls `keytool` with `-keyalg ML-DSA -sigalg ML-DSA-65` to produce PKCS12 keystores and truststores under `local/pq-shp-stores/`.

1.5 Tests and Results

`PqShpTest` runs end-to-end handshakes for both variants across three cipher suites (AES/GCM, ChaCha20-Poly1305, AES/CBC + HMAC-SHA256), verifying that the negotiated suite, requested movie, and UDP port are correctly conveyed. All tests pass.

`PqShpHandshakeTimingTest` runs 10 complete handshakes per variant per suite, comparing Classic (ECDSA + ECDH), Hybrid (ML-DSA + ECDH), and Full PQ (ML-DSA + ML-KEM) latencies. Representative results are shown in Table 1.

Variant	Cipher Suite	Min (ms)	Avg (ms)	Max (ms)
Classic (ECDSA + ECDH)	AES/GCM/NoPadding	56	79	102
Classic (ECDSA + ECDH)	ChaCha20-Poly1305	54	70	101
Classic (ECDSA + ECDH)	AES/CBC + HMAC-SHA256	57	79	105
Hybrid (ML-DSA + ECDH)	AES/GCM/NoPadding	64	79	125
Hybrid (ML-DSA + ECDH)	ChaCha20-Poly1305	60	86	104
Hybrid (ML-DSA + ECDH)	AES/CBC + HMAC-SHA256	56	76	107
Full PQ (ML-DSA + ML-KEM)	AES/GCM/NoPadding	55	80	106
Full PQ (ML-DSA + ML-KEM)	ChaCha20-Poly1305	55	78	100
Full PQ (ML-DSA + ML-KEM)	AES/CBC + HMAC-SHA256	57	91	103

Table 1: Handshake latency over 10 iterations per variant per suite (loopback).

All three variants show similar latency (55–105 ms). The averages are within measurement noise, indicating that TCP and serialisation overhead dominate over the asymmetric operations. The symmetric cipher suite has no effect in any variant.

2 Limitations and Assumptions

The trust model is unchanged from Parts 1 and 2. ML-KEM operations use Bouncy Castle 1.84 rather than the JDK native provider, since the JDK does not yet expose a `KeyAgreement`-compatible ML-KEM API suitable for our use pattern. ML-DSA uses the JDK native provider (JEP 497, Java 25), which is incompatible with Bouncy Castle's own ML-DSA signing path; the two cannot be mixed. Certificate revocation is not implemented.